GraphOne / FrontierAtlas Intelligence Graph

Technical Architecture & Scale Strategy Document (Targeting 500,000+ Records)

Executive Summary: This document outlines the architectural blueprint for GraphOne's autonomous, production-grade data ingestion pipeline. It addresses high-concurrency collection of 500,000+ entities (Startups, Products, Research Papers with GitHub metrics, Jobs, and News), robust multi-tier LLM fallback orchestration, distributed 24-hour freshness tracking, and unified graph-vector storage.

1. Scale Strategy: Acquiring 500,000+ Entities

To scale ingestion from initial thousands to over 500,000 records without manual intervention, we decouple web crawling from structured extraction through an asynchronous, event-driven architecture.

Architectural Components:

- **Distributed Async Crawler Pool:** Built on Python's asyncio + aiohttp / Playwright Async nodes, deployed across Kubernetes Pods. Scrapers run stateless worker loops reading targeted domains from a centralized Redis Queue.
- **Distributed Rate Limiting & Proxy Mesh:** Integrates BrightData / Smartproxy rotating IP networks with token-bucket rate limiters in Redis. Prevents IP bans and handles anti-bot measures (Cloudflare, Datadome) via TLS fingerprinting (curl_cffi) and browser-based stealth proxies.
- **Source Vertical Parallelization:** Dedicated workers partition work by vertical: arXiv / PapersWithCode API dumps for research papers, ProductHunt / Crunchbase sitemaps for startups and products, and RSS / Career portals for 24-hr signals.

Vertical	Target Volume	Primary Ingestion Source	Concurrency Mechanism
Startups	150,000+	Crunchbase, YC, PitchBook	Async HTTP + Sitemap Walkers
Products	150,000+	ProductHunt, G2, App Store	GraphQL API + Async Workers
Research Papers	200,000+	arXiv OAI-PMH, PapersWithCode	Bulk S3 Dumps + GitHub REST API

2. Resilient LLM Integration: Managing 413s & 429s

Extracting schema-compliant JSON from raw, unstructured web pages requires managing strict API quotas and token window limits. Our system employs a multi-tiered fallback pipeline with adaptive payload chunking.

Handling Context Window Overflows (413 Payload Too Large):

- **Semantic Truncation & DOM Distillation:** Raw HTML is stripped of scripts, styles, and boilerplate navigation using BeautifulSoup4 and html2text. If payload exceeds tier limits, a density-preserving chunker retains 70% header/main content and 30% tail context.
- **Recursive Map-Reduce Chunking:** For massive articles or technical papers, text is split into 4k token overlapping windows, processed independently, and merged via a final synthesis pass.

Handling Rate Limits (429 Too Many Requests):

- **Multi-Tier Fallback Chain:** Gemini 1.5 Flash (Primary) → Groq Llama 3.3 70B (Tier 2) → DeepSeek V3 (Tier 3). If primary tier returns 429 or times out, request cascades automatically.
- **Full Jitter Exponential Backoff:** Backoff time calculated as Wait = Random(0, Min(Cap, Base * 2^attempt)) to prevent thundering herd problems across worker pods.

LLM Tier	Model	Context Window	Primary Role
Primary	Gemini 1.5 Flash	1,000,000 Tokens	High-throughput bulk extraction
Fallback 1	Groq Llama 3.3 70B	128,000 Tokens	Low-latency fallback on Gemini 429
Fallback 2	DeepSeek V3	64,000 Tokens	High-reasoning final extraction

3. Freshness Tracking & Anti-Deduplication Engine

GraphOne guarantees real-time signal accuracy by validating that news and job postings are published within the last 24 hours.

- **Bloom Filter + Redis Fingerprinting:** Every ingested item URL and content hash (SHA-256) is checked against a Redis Bloom Filter. If present, crawler skips processing immediately.
- **Date Normalization Engine:** Custom logic parses ISO dates, Unix timestamps, and relative strings ('3 hours ago', '1 day ago'). Items with publication dates older than 24 hours are dropped prior to LLM extraction.
- **Intelligent Content Difference Heuristics:** When strict publication meta tags are absent, a lightweight DOM hashing comparison checks for new semantic content blocks since the previous crawl pass.

4. Deterministic Entity Resolution Pipeline

Entity ambiguity (e.g. 'OpenAI', 'OpenAI Inc.', 'Open AI') compromises intelligence graph integrity. We implement a multi-stage deterministic canonicalization engine:

1. **Legal Suffix Normalization:** Strip corporate suffixes (Inc., LLC, Corp, Labs, AI) and standardize whitespace.
2. **Seed Database Index Lookup:** Match normalized strings against an in-memory seed index of 50+ known canonical AI organizations.
3. **Fuzzy Levenshtein & Jaro-Winkler Matching:** High-confidence fuzzy string matching (threshold > 0.88) clusters minor spelling variations.
4. **Audit Logging:** Every canonicalization event is recorded in the *Entity Mapping Log* with raw string, canonical output, confidence score, and method.

5. Storage Architecture: Graph & Vector Hybrid Model

Mapping multi-dimensional relationships between Startups, Founders, Products, Papers, and Jobs requires a dual-database storage strategy:

- **Primary Relational DB (PostgreSQL + PostGIS):** Stores raw entity records, normalized schemas, audit logs, and transactional job/news signals with ACID guarantees.
- **Graph Database (Neo4j / Amazon Neptune):** Represents canonical nodes (Startup, Product, Paper, Founder) and edge relationships (PRODUCES, AUTHORED_BY, HIRING_FOR, CITES). Allows sub-millisecond graph

traversal queries.

• **Vector Database (Qdrant / Pinecone):** Stores 1536-dimensional embeddings (e.g. OpenAI text-embedding-3-small) of full news text and paper abstracts to enable semantic search and automatic entity disambiguation.

Storage Layer	Technology	Purpose & Data Models
Primary Database	PostgreSQL 16	Structured entity records, JSONB schemas, system logs
Graph Database	Neo4j / Neptune	Complex entity-relationship mapping & graph queries
Vector Database	Qdrant Cloud	Semantic search, paper/news embeddings, similarity clustering
In-Memory Cache	Redis Enterprise	Bloom filters, deduplication hashes, distributed queue